LAB # 9:

K-NEAREST NEIGHBORS CLASSIFICATION MODEL

Objectives:

- To implement KNN on a given dataset

Hardware/Software Required:

Hardware: Desktop/ Notebook Computer

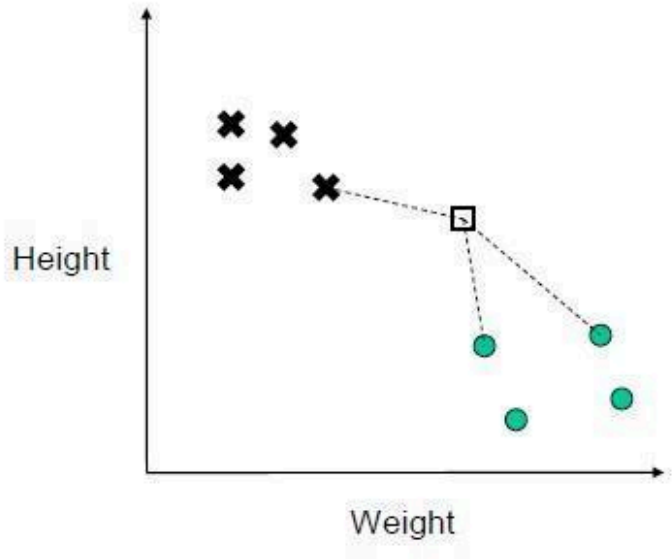
Software Tool: Anaconda3

Introduction:

K-Nearest Neighbors:

In pattern recognition and machine learning, k-nearest neighbors (KNN) is a simple algorithm that stores all available cases and classifies new cases based on a similarity measure (e.g. distance). KNN is a non-parametric method where the input consists of the k closest training examples in the feature space. The output is a class membership. An object is classified by a majority vote of its neighbors, with the object being assigned to the class most common among its k nearest neighbors (k is a positive integer, typically small). If k = 1, then the object is simply assigned to the class of that single nearest neighbor.

1. *Measure distance to all points*
2. *Find closest "k" points* ← **(here k=3, but it could be more)**



Euclidean distance

$$d(p,q) = \sqrt{(q_1 - p_1)^2 + (q_3 - p_3)^2 + \cdots + (q_n - p_n)^2}$$

$$= \sqrt{\sum_{i=1}^{n}(q_i - p_i)^2}$$

Where **n** is number of features

Example dataset:

15	95	1
33	90	1
78	70	0
70	45	0
80	18	0
35	65	1

45	70	1
31	61	1
50	63	1
98	80	0
73	81	0
50	18	0

The first table is the training set while the other is the test dataset.

Steps in KNN

In KNN we must basically perform three steps

- First, we must find distance of test data Row with all the training set rows
- Find the K nearest neighbors i.e., select those training set rows for which test data row has min distance
- Choose the maximum occurring class from those neighbor rows

Pseudocode

Step 1: Writing a function to calculate the distance between two data values

```
Function Euclidian_distance (row1, row2):

#where row1 and row2 can be any two rows   #row1 = > training row2 => testing
```

```
        #here all rows are treated as list

    col= Len(row1) #it returns the number of columns in a row    col =2

        loop form 0 to col – 1
                Calculate distance
        Return distance

#checking for only 1 row right now, but later, to predict the values for whole the test data

Row1= dataset [0]     #picks the first row from the dataset

For rows in a dataset:    #loop for all values in sample of row1
      Call and print the Euclidian_distance function and pass these arguments (i.e.  Row1 and rows)
```

Step2: Finding K nearest neighbors with minimum distance

```
Function get_neighbors (trainingSet, testingRow,k_number):

        distanceList= []
         Loop for all rows in the training set and maintain index, i.e. row number as well:

                distSaving= eucledian_distance(arg1, arg2)  #calling the function defined above

                    Now append the above [ distSaving] in another list, let's call it 'distanceList'

               print (distanceList)
```

```
                Sort the distanceList

        Print(distanceList)

        K_neighbors=sorted_list[0:k]

        Return K_neighbors

testingRow= row1 #selecting the first row of the data as test for right now

print and call the get_neighbors(dataset, testingRow, 3)
```

getting the maximum occurring class,

```
Function predict_classification(trainingSet, testingRow, k_number):

        #call the above defined function here so now kn_neighbor will have neighbor list

        Kn_neighbors=  get_neighbors(traingSet, testingRow, k_number)

        Class_values=[]

        Loop till all elements of kn_neighbors:

                Append the last value/column of every list/row from the kn_neighbors list in the
                class_values list

        #now max occurring list will be selected based on count

        pred= max(set(class_values), key=class_values.count)

        return pred
    #call the function and print the output
```

Tasks:

1. Implement KNN in python for the sample dataset given below (**using first row as testing row, and the sample dataset is treated as the training Set)**

   Hint (Test sample = dataset [0] and trainset = [1:])

X1	X2	Class
2.7810836	2.550537003	0
1.465489372	2.362125076	0
3.396561688	4.400293529	0
1.38807019	1.850220317	0
3.06407232	3.005305973	0
7.627531214	2.759262235	1
5.332441248	2.088626775	1
6.922596716	1.77106367	1
8.675418651	-0.242068655	1
7.673756466	3.508563011	1

```
##Importing dataset
Dataset = [[2.781026.2.550537003,0],
                [….],
                 […],
                 […],
                 [7.673756466,35.08563011,1]]
```

2. Using the above code, predict class for all the rows of the sample dataset, and find Accuracy of the algorithm (where accuracy = correct/total * 100) (Hint: compare the predicted and original label for each row)

3. Download the csv file given, load it (as given below) and then make prediction for the first row of this csv data

```
import csv
file = open('fruit_data_with_colours.csv')
```

```
csvreader = csv.reader(file)
dataset = []
for row in csvreader:
      dataset. append(row)

dataset.remove(dataset[0])
dataset = np.array(dataset, dtype=float)
```